Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Programación Python con Aplicaciones en el Ámbito Científico

Profesor Alan Badillo Salas

Proyectos finales

Proyecto final con aplicaciones científicas

Semana 5. Viernes 9, octubre 2025.

Introducción

Durante el curso, se han desarrollado los diferentes temas planeados en el contenido del curso, hemos visto la programación en Python, la manipulación y visualización de los datos, algunas aplicaciones científicas y temas más profundos como la programación orientada a objetos y las redes neuronales.

Con este conocimiento acumulado, tenemos la capacidad suficiente para desarrollar un proyecto completo de forma individual, para demostrar y acreditar los conocimientos que hemos adquirido durante todo el curso, que aunque en principio parezcan dispersos, al guiarlos en una práctica se verá más natural y evidente el conocimiento que se ha acumulado.

En estos **Proyectos finales**, dominaremos un análisis completo para generar un producto final aplicable en el ámbito científico y otras áreas. De estos proyectos, seleccionaremos uno que sea el de más interés y lo desarrollaremos de principio a fin, para generar un documento que exponga el proyecto y una presentación para presentarlo en la clase final.

Elige uno de los proyectos propuestos y replíca el proyecto que se describe de forma guiada. En un documento **PDF** con una portada breve la identidad del *Instituto Politécnico Nacional*, el *Centro de Investigación en Computación* y el *Departamento de diplomados y extensión profesional*, con tu nombre completo, correo, así como lugar y fecha, documenta los resultados del proyecto y envíalos por correo a **badillosalas@outlook.com** con el título **PyCien 2026 - Proyecto Final**. No es necesario poner cuerpo en el correo o adjuntar más archivos, solo el documento PDF es suficiente. El documento PDF se debe llamar *pycien26_SIGLAS.pdf* con las SIGLAS formadas por tu nombre y apellidos. Opcionalmente, genera una presentación exponiendo solamente los resultados de tu proyecto.

1. Proyecto 1 - La Criba de Eratóstenes

La Criba de Eratóstenes es un procedimiento para generar los primeros N números primos, consiste en tener una lista con los números naturales de 0 a N que en cada posición guarde el valor lógico $Verdadero$ y $Falso$ para indicar si el número natural es primo o no lo es.

El procedimiento es sencillo, inicialmente todos los números naturales se suponen primos en cada posición, y luego los múltiplos se desactivan ya que no pueden ser primos, porque son múltiplos del primo anterior, así al inicio 2 es primo, pero $4, 6, 8, 10, ...$ ya no pueden ser primos, por lo que se marcan en $Falso$ sobre la criba, el siguiente natural es 3, como es primo entonces sus múltiplos ya no pueden ser primos, por lo que $6, 9, 12, 15, ...$ se ponen en $Falso$ sobre la criba, el siguiente es 4 pero ya fue marcado como no primo y se continua, luego 5, este no fue marcado, así que se considera primo, pero $10, 15, 20, 25, 30, ...$ se ponen en $Falso$ sobre la criba.

Para implementar este algoritmo, se requiere una lista, que inicialmente debe ser llenada con valores $Verdadero$, para cada posición que representa el número natural que se quiere saber si es o no un número primo.

Luego, para cada número n, se comprueba si la criba en la posición n es verdadera (siguie siendo primo), y en tal caso se desactivan todos sus múltiplos entre $n + n$ y N.

Para resolver este proyecto, implementa el siguiente pseudocódigo, intenta hacer su diagrama de flujo para complementarlo.

```
1. INICIO
2. DECLARA VARIABLE N <- 100
3. DECLARA LISTA VACÍA criba <- []
4. PARA CADA n EN RANGO(0, N + 1), HACER
    4.1 AGREGAR A LISTA criba VALOR VERDADERO
5. PARA CADA n EN RANGO(2, N), HACER
    5.1 RECUPERAR esPrimo <- criba[n]
    5.2 SI esPrimo, ENTONCES
        5.2.1 PARA m EN RANGO(n + n, N, n), ENTONCES
            5.2.1.1 MODIFICAR criba[m] <- FALSO
6. PARA CADA n EN RANGO(2, N), HACER
    6.1 RECUPERAR esPrimo <- criba[n]
    6.2 IMPRIMIR n, esPrimo
7. FIN
```

El algoritmo comienza llenando una criba que supone todos los números de 0 a N como primos, sin embargo, a partir del 2 se hace el cálculo de los múltiplos, para 2, 3, ..., N se verifica si son primos, en caso de serlo, se desactivan sus primos y se continúa así para cada número $n \leq N$.

2. Proyecto 2 - Canastas de frutas

Un objeto puede envolver un contexto de variables y operaciones que se coordinan para mantener un estado aislado y encapsulado. Esto permite por ejemplo mantener una lista de datos y sus operaciones de manera coordinada para representar objetos análogos al mundo real, como podrían ser clientes, productos o carritos de compra. La versión más sencilla de un carrito de compras consiste en una canasta de productos, la cual puede almacenar y quitar productos o transferirlas de una canasta a otra.

En este proyecto se modelará una canasta de frutas que permita consultar, agregar, quitar y transferir frutas entre varias canastas. Cada canasta tendrá su propio nombre exigido en la inicialización y comenzará vacía. Se agregarán métodos que permitan describir de diferente manera la canasta, por ejemplo, ver la lista de frutas, contar cuántas frutas distintas hay, agregar, quitar y transferir una fruta si es posible.

Implemente en cada celda las definiciones y pruebas para observar cómo opera la canasta de frutas. Finalmente, documente el código para marcar explícitamente que hace cada línea de código y realice pruebas adicionales que considere interesantes.

Algunas mejoras al proyecto:

1. Guardar las operaciones que ocurren al agregar, quitar y transferir en una lista interna de operaciones
2. Describir las operaciones que ocurrieron sobre la canasta
3. Transferir todas las frutas de una canasta a otra
4. Quitar todas las frutas repetidas
5. Agregar una fruta de cada una de las que ya se tienen en la canasta
6. Transferir todas las frutas de una misma fruta a otra canasta
7. Transferir todas las frutas de la otra canasta
8. Devolver el número de frutas de una fruta
9. Devolver el número total de frutas
10. Limitar el máximo de cada fruta
11. Limitar el máximo de todas las frutas totales

```
class Canasta:

    nombre = None
    frutas = None

    def __init__(self, nombre):
        self.nombre = nombre
        self.frutas = []

    def agregar(self, fruta):
        self.frutas.append(fruta)

    def quitar(self, fruta):
        if not fruta in self.frutas:
            print(f"LA FRUTA {fruta}")
            print(f"NO ESTÁ EN LA CANASTA {self.nombre}")
            print()
            return False
        else:
            self.frutas.remove(fruta)
            return True

    def transferir(self, canastaOtra, fruta):
        if self.quitar(fruta):
            canastaOtra.agregar(fruta)
        else:
            print("LA FRUTA NO SE PUDO TRANSFERIR DE:")
            print(f"CANASTA {self.nombre}")
            print(f"A {canastaOtra.nombre}")
            print()

    def describirTodas(self):
        print(f"CANASTA {self.nombre}")
        print("-" * 60)
        for indice, fruta in enumerate(self.frutas):
            print(f"{indice + 1}. {fruta}")
        print()

    def describirConteo(self):
        print(f"CANASTA {self.nombre}")
        print("-" * 60)
        frutasConteo = {}
        for fruta in self.frutas:
            if not fruta in frutasConteo:
                frutasConteo[fruta] = 1
            else:
                frutasConteo[fruta] += 1
        for fruta, conteo in frutasConteo.items():
            print(f"{fruta:>10s} x{conteo}")
        print()
```

```
canasta1 = Canasta("Canasta 1")
canasta2 = Canasta("Canasta 2")
canasta3 = Canasta("Canasta 3")
```

```
canasta1.agregar("MANZANA")
canasta1.agregar("MANZANA")
canasta1.agregar("MANZANA")
canasta1.agregar("PERA")
canasta1.agregar("PERA")
canasta1.agregar("MANGO")

canasta1.describirTodas()
```

```
canasta1.describirConteo()
```

```
canasta2.agregar("KIWI")

canasta2.describirConteo()
```

```
canasta3.agregar("GUAYABA")
canasta3.agregar("GUAYABA")
canasta3.agregar("GUAYABA")
canasta3.quitar("GUAYABA")
canasta3.quitar("MANZANA")

canasta3.describirTodas()
```

```
canasta1.transferir(canasta2, "MANZANA")
canasta1.transferir(canasta2, "MANGO")

canasta1.describirTodas()
```

```
canasta2.describirTodas()
```

```
canasta1.transferir(canasta3, "MANGO")

canasta1.describirTodas()
```

```
canasta2.transferir(canasta3, "MANGO")

canasta2.describirTodas()
```

```
canasta3.describirTodas()
```

3. Proyecto 3 - Estadística descriptiva sobre la Supervivencia de los Pasajeros del Titanic

Para entender la estadística que describe un conjunto de datos necesitamos reconocer cada característica y separar las características informativas de las respuestas, es decir, las características que informan las respuestas, este proceso se conoce como caracterización.

Una vez caracterizadas las variables del conjunto de datos podemos analizar individualmente cada característica y también analizar su comportamiento frente a cada respuesta.

En el conjunto de datos del Titanic, se tienen diferentes características como la edad, el sexo, la clase del pasajero, la tarifa que pagó, la cantidad de familiares con los que viajó, etc.

Dependiendo el tipo de característica, si es numérica o categórica, analizaremos de forma individual su comportamiento frente a la respuesta.

Análisis individual:

1. **Categórica** - Conteo las observaciones de cada categoría, esto se conoce como el soporte o número de observaciones de cada categoría. También la proporción de cada categoría dado el total de observaciones.
2. **Numérica** - Media, Desviación Estándar, Mínimo, Cuartil 1 (Q_1 al 25 % ordenado), Cuartil 2 (Q_2 al 50 % ordenado), Cuartil 3 (Q_3 al 75 % ordenado), Máximo

Las estadísticas del análisis individual nos dirán cuál es la probabilidad o posibilidad de encontrar un valor dentro del intervalo o categorías según corresponda.

Análisis contra una respuesta categórica:

1. **Categórica** - Como ambas características son categóricas (la variable y la respuesta), generaremos la matriz de conteos donde ambos existen y la matriz de proporciones entre ambas.
2. **Numérica** - Como la respuesta es categórica, analizaremos cómo se comportan los estadísticos de la variable para cada categoría de la respuesta y si estos se comportan de manera similar o diferente entre cada grupo (categoría).

Análisis contra una respuesta numérica:

1. **Categórica** - Como la respuesta es numérica, analizaremos cómo se comportan los estadísticos de la variable para cada categoría de la variable y si estos se comportan de manera similar o diferente entre cada grupo (categoría).

2. **Numérica** - Como ambas características son numéricas (la variable y la respuesta), mediremos la correlación entre ambas.

En el conjunto de datos del titanic ubicaremos 2 respuestas:

1. **Supervivencia** - Categórica con dos clases 1 si sobrevive y 0 si no sobrevive.

2. **Clase del Pasajero** - Numérica, que aunque también se podría ver como categórica de 3 clase 1, 2 y 3 se puede estudiar cómo incrementa o decrementa la clase del pasajero de manera continua.

Lista de características informativas:

1. **Edad** - Numérica, algunos valores están perdidos y se deben imputar.

2. **Sexo** - Categórica de 2 clases $male$ y $female$.

3. **Total de familia** - Numérica, se construye de sumar la familia lateral de hermanos y cónyuges y la familia vertical de padres e hijos.

4. **Tarifa logarítmica** - Numérica, derivada del logaritmo más 1 de la tarifa, ya que esta contine muchos puntos atípicos.

5. **Cabina** - Categórica de 2 clases 1 y 0 se construye con la información de si hay cabina.

6. **Embarcación** - Categórica de 3 clases Q y S, C referente a puertos en Irlanda, Londres y Francia.

Determina las estadísticas individuales para cada característica informativa y las estadísticas contra las respuestas siguiendo los ejemplos.

```
# Codificar una característica numérica a la que le faltan
# datos, con imputación a la mediana
edad = titanic["Age"].fillna(titanic["Age"].median())

# Codificar una característica numérica como resultado
# de la suma de otras dos
familia = titanic["SibSp"] + titanic["Parch"]

# Codificar una característica binaria a dos clases
# formalizadas
supervivencia = titanic["Survived"].map({
    1: "SOBREVIVE",
    0: "NO SOBREVIVE"
})
```

```
# Codificar una característica categórica de dos clases
# a otras dos clases renombradas
sexo = titanic["Sex"].map({
    "male": "HOMBRE",
    "female": "MUJER"
}).fillna("DESCONOCIDO")

# Codificar una característica numérica con transformación
# del logaritmo para reducir los puntos atípicos
tarifa = titanic["Fare"].apply(numpy.log1p)

# Codificar una variable categórica a partir de
# identificar si es o no un valor nulo
cabina = (~titanic["Cabin"].isna()).astype(int).map({
    1: "CON CABINA",
    0: "SIN CABINA"
})

# Codificar una característica categórica de tres clases
# a otras tres clases renombradas
embarcacion = titanic["Embarked"].map({
    "Q": "IRLANDA",
    "S": "LONDRES",
    "C": "FRANCIA"
})
```

```
# Cambiar los nombres de las series
supervivencia.name = "Supervivencia"
clase_pasajero.name = "Clase Pasajero"
edad.name = "Edad"
sexo.name = "Sexo"
familia.name = "Familia"
tarifa.name = "Tarifa"
cabina.name = "Cabina"
embarcacion.name = "Embarcacion"
```

Estadística individual del un eje categórico

```
pandas.concat([
    supervivencia.value_counts(),
    supervivencia.value_counts(normalize=True),
], axis=1)
```

Estadística individual del un eje numérico

```
pandas.DataFrame(clase_pasajero.describe())
```

Estadística de contraste entre un eje categórico y uno numérico

```
pandas.DataFrame({
    "Supervivencia": supervivencia,
    "Edad": edad
}).groupby(["Supervivencia"]).describe()
```

Estadística de contraste entre un eje categórico y otro categórico

```
# Soporte
A = pandas.DataFrame({
    "Supervivencia": supervivencia,
    "Sexo": sexo
}).groupby(["Supervivencia", "Sexo"]).size().unstack()

# Propociones verciales
B = A.divide(A.sum(axis=0), axis=1)

# Propociones horizontales
C = A.divide(A.sum(axis=1), axis=0)
```

Determina todas las estadísticas individuales para cada característica del conjunto de datos del titanic y todos los contrastes con la respuesta de Supervivencia.

4. Proyecto 4 - Visualización de datos sobre la Supervivencia de los Pasajeros del Titanic

Para entender un conjunto de datos a través de lo que visualizan sus datos necesitamos reconocer cada característica y separar las características informativas de las respuestas, es decir, las características que informan las respuestas, este proceso se conoce como caracterización.

Una vez caracterizadas las variables del conjunto de datos podemos analizar individualmente cada característica y también analizar su comportamiento frente a cada respuesta.

En el conjunto de datos del Titanic, se tienen diferentes características como la edad, el sexo, la clase del pasajero, la tarifa que pagó, la cantidad de familiares con los que viajó, etc.

Dependiendo el tipo de característica, si es numérica o categórica, visualizaremos de forma individual su comportamiento frente a la respuesta.

Visualización individual:

1. **Categórica** - Barras que muestren el soporte o proporción de cada categoría, se visualiza cuántas veces crece la barra más pequeñas para completar las más grandes y se imagina la proporción de cada clase.
2. **Numérica** - Gráfico combinado de la densidad y la caja de cuartiles, se visualiza las zonas donde hay mayor concentración de los datos y si estos forman una sola montaña de verdad.

Las visualizaciones individuales nos dirán cuál es la probabilidad o posibilidad de encontrar un valor dentro del intervalo o categorías según corresponda.

Visualización contra una respuesta categórica:

1. **Categórica** - Mapa de color/calor en tres momentos: el soporte, la probabilidad vertical y la probabilidad horizontal, se busca ver si la concentración se separa en las categorías o se mantiene uniforme para cualquier zona.
2. **Numérica** - Grafico combinado de la separación de las cajas de cuartiles y las densidades por cada categoría, se busca visualizar si cambia la región de concentración para cada categoría.

Visualización contra una respuesta numérica:

1. **Categórica** - Grafico combinado de la separación de las cajas de cuartiles y las densidades por cada categoría, se busca visualizar si cambia la región de concentración para cada categoría.

2. **Numérica** - Gráfico combinado entre dispersión de puntos, caja cuartil derecha y caja cuartil superior (para la otra característica numérica), se busca encontrar un patrón lineal entre los datos que indique una tendencia entre ambas características.

En el conjunto de datos del titanic ubicaremos 2 respuestas:

1. **Supervivencia** - Categórica con dos clases 1 si sobrevive y 0 si no sobrevive.
2. **Clase del Pasajero** - Numérica, que aunque también se podría ver como categórica de 3 clase 1, 2 y 3 se puede estudiar cómo incrementa o decrementa la clase del pasajero de manera continua.

Lista de características informativas:

1. **Edad** - Numérica, algunos valores están perdidos y se deben imputar.
2. **Sexo** - Categórica de 2 clases *male* y $female$.
3. **Total de familia** - Numérica, se construye de sumar la familia lateral de hermanos y cónyuges y la familia vertical de padres e hijos.
4. **Tarifa logarítmica** - Numérica, derivada del logaritmo más 1 de la tarifa, ya que esta contine muchos puntos atípicos.
5. **Cabina** - Categórica de 2 clases 1 y 0 se construye con la información de si hay cabina.
6. **Embarcación** - Categórica de 3 clases Q y S, C referente a puertos en Irlanda, Londres y Francia.

Determina las estadísticas individuales para cada característica informativa y las estadísticas contra las respuestas siguiendo los ejemplos.

```
# Codificar una característica numérica a la que le faltan
# datos, con imputación a la mediana
edad = titanic["Age"].fillna(titanic["Age"].median())

# Codificar una característica numérica como resultado
# de la suma de otras dos
familia = titanic["SibSp"] + titanic["Parch"]

# Codificar una característica binaria a dos clases
# formalizadas
supervivencia = titanic["Survived"].map({
    1: "SOBREVIVE",
    0: "NO SOBREVIVE"
})
```

```
# Codificar una característica categórica de dos clases
# a otras dos clases renombradas
sexo = titanic["Sex"].map({
    "male": "HOMBRE",
    "female": "MUJER"
}).fillna("DESCONOCIDO")

# Codificar una característica numérica con transformación
# del logaritmo para reducir los puntos atípicos
tarifa = titanic["Fare"].apply(numpy.log1p)

# Codificar una variable categórica a partir de
# identificar si es o no un valor nulo
cabina = (~titanic["Cabin"].isna()).astype(int).map({
    1: "CON CABINA",
    0: "SIN CABINA"
})

# Codificar una característica categórica de tres clases
# a otras tres clases renombradas
embarcacion = titanic["Embarked"].map({
    "Q": "IRLANDA",
    "S": "LONDRES",
    "C": "FRANCIA"
})
```

```
# Cambiar los nombres de las series
supervivencia.name = "Supervivencia"
clase_pasajero.name = "Clase Pasajero"
edad.name = "Edad"
sexo.name = "Sexo"
familia.name = "Familia"
tarifa.name = "Tarifa"
cabina.name = "Cabina"
embarcacion.name = "Embarcacion"
```

Visualización individual del un eje categórico

```
seaborn.barplot(supervivencia.value_counts())
```

Visualización individual del un eje numérico

```
figure, axes = pyplot.subplots(2, 1, height_ratios=(1, 5))
seaborn.boxplot(x=edad, fill=False, ax=axes[0])
seaborn.kdeplot(x=edad, fill=True, ax=axes[1])
axes[0].set_xlabel(None)
```

Visualización de contraste entre un eje categórico y uno numérico

```
figure, axes = pyplot.subplots(2, 1, figsize=(20, 10))

seaborn.boxplot(x=edad, y=supervivencia, hue=supervivencia,
                fill=False, palette="Set1", ax=axes[0])
seaborn.kdeplot(x=edad, hue=supervivencia, fill=True,
                palette="Set1", ax=axes[1])
```

Visualización de contraste entre un eje categórico y otro categórico

```
A = pandas.DataFrame({
    "Supervivencia": supervivencia,
    "Sexo": sexo
}).groupby(["Supervivencia", "Sexo"]).size().unstack()

B = A.divide(A.sum(axis=0), axis=1) * 100
C = A.divide(A.sum(axis=1), axis=0) * 100

figure, axes = pyplot.subplots(1, 3, figsize=(30, 10))

seaborn.heatmap(A, cmap="Blues",
                annot=True, fmt=".0f", ax=axes[0])
seaborn.heatmap(B, cmap="Reds",
                annot=True, fmt=".1f", ax=axes[1])
seaborn.heatmap(C, cmap="Purples",
                annot=True, fmt=".1f", ax=axes[2])
```

Determina todas las visualizaciones individuales para cada característica del conjunto de datos del titanic y todos los contrastes con la respuesta de Supervivencia.

5. Proyecto 5 - Red neuronal de Supervivencia de los Pasajeros del Titanic

Las redes neuronales son el cúlmino de los modelos de *Machine Learning* para aprender respuestas abiertas (lineales) y cerradas (logísticas), a través de una sola neurona la red neuronal funciona como el modelo de machine learning, sin embargo, cuando agregamos capas de neuronas, el aprendizaje comienza a demostrar mayor potencia.

El flujo para analizar un conjunto de datos mediante una red neuronal es:

1. Separar las características informativas en una representación numérica, ya sea directa o mediante *dummies* en la codificación *One-Hot* (seleccionar las *dummies* informativas ignorando la *dummy* de la categoría base).

2. Aplicar el modelo de aprendizaje, determinar cuántas características informan y si la salida es abierta o cerrada, para construir un modelo de red neuronal, asociado a un optimizador como *SGD - Stochastic Gradient Descend / Descenso del Gradiente Estocástico*, *ADAM - Adaptative Momentum / Descenso del Gradiente con Momento Adaptativo*, etc., una función de pérdida: *Lineal - MSE / Mean Square Error / Error Cuadrático Medio* o *Logítica - Binary Cross Entropy / Entropía Cruzada Binaria* y métricas de desempeño adicionales como *Lineal - MAE / Mean Absolute Error / Error Absoluto Medio* o *Logístico - Accuracy / Exactitud* para compilarlo y entrenarlo durante varias épocas.

3. Evaluación del aprendizaje, a través de la caída en el error de la pérdida y el aumento en el desempeño de la métrica observada.

Y finalmente, se tendrá una red neuronal entrenada que se puede almacenar en físico para poder hacer predicciones cada que se requiera.

Primero instalaremos las librerías necesarias:

```
# Verifica que la versión del kernel coincida
! python -V
# Instala las librerías necesarias
! python -m pip install numpy pandas
! python -m pip install matplotlib seaborn
! python -m pip install tensorflow keras
```

Luego importaremos las librerías:

```
import numpy
import pandas

from matplotlib import pyplot
import seaborn

import keras
```

Ahora construiremos la matriz X de entrenamiento

```
# Edad numérico
x1 = edad
# Sexo categórico (2 dummies)
# La base es ser hombre
x2 = (sexo == "MUJER").astype(int)
# Familia numérico
x3 = familia
# Tarifa numérico
x4 = tarifa
# Cabina categórico (2 dummies)
# La base es no tener cabina
x5 = (cabina == "CON CABINA").astype(int)
# Embarcación categórico (3 dummies)
# La base es embarcar en londres
x6 = (embarcacion == "IRLANDA").astype(int)
x6.name = "Embarca Irlanda"
# La base es embarcar en londres
x7 = (embarcacion == "FRANCIA").astype(int)
x7.name = "Embarca Francia"

X = pandas.DataFrame([x1, x2, x3, x4, x5, x6, x7]).T

X
```

Y el vector de la respuesta de supervivencia:

```
y1 = (supervivencia == "SOBREVIVE").astype(int)

y1
```

Después definimos el modelo de la red neuronal, primero usa una sola neurona de respuesta y luego descomenta las capas ocultas para ver si mejora con más neuronas el aprendizaje, agrega o cambia el número de capas y cantidad de neruronas:

```
# Red neuronal de 1 sola neurona / ML Logístico
modelo = keras.Sequential([
    keras.Input(shape=(7,)),
    # 64 neuronas primera capa
    #keras.layers.Dense(64, activation="relu"),
    # 64 neuronas segunda capa
    #keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid")
])

modelo.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

modelo.summary()
```

Entrena el modelo durante 500 épocas o más si consideras necesario (observa si el error puede seguir bajando o ya no y si el aprendizaje puede seguir subiendo o ya no). Recuperaremos también el historial del entrenamiento:

```
historial = modelo.fit(X, y1, epochs=500)
```

Gráfica del error y la exactitud para ver si el error puede bajar más o si el aprendizaje puede subir más:

```
epocas = historial.epoch
error = historial.history["loss"]
precision = historial.history["accuracy"]

figure, axes = pyplot.subplots(2, 2, figsize=(20, 20))

seaborn.lineplot(x=epocas, y=error, ax=axes[0][0])
seaborn.boxplot(y=error[-20:], showfliers=False,
                fill=False, ax=axes[0][1])
seaborn.lineplot(x=epocas, y=precision, ax=axes[1][0])
seaborn.boxplot(y=precision[-20:], showfliers=False,
                fill=False, ax=axes[1][1])
```

Guarda el modelo entrenado de la red neuronal, según las neuronas que tenga, por ejemplo, *nn_titanic_64_64_1.keras*

```
modelo.save("nn_titanic_1.keras")
```

Carga un modelo de la red neuronal almacenada para evaluarlo y obtener predicciones:

```
modelo = keras.models.load_model("nn_titanic_1.keras")

modelo.summary()
```

Crea predicciones personalizadas con supuestos de qué pasaría en un pasajero con ciertas características y variaciones de este:

```
modelo.predict(numpy.array([
    # 23 años | MUJER | 0-FAMILIA | 28 libras |
    # CON CABINA | NO IRLANDA | NO FRANCIA
    [23, 1, 0, numpy.log1p(28), 1, 0, 0],
    # 23 años | HOMBRE | 0-FAMILIA | 28 libras |
    # CON CABINA | NO IRLANDA | NO FRANCIA
    [23, 0, 0, numpy.log1p(28), 1, 0, 0],
]))
```

Y con esto ya tenemos una red neuronal entrenada y almacenada a la que le podemos pedir predicciones en cualquier momento solo cargando el archivo.